DLA - Lab. 2

Lapo Bisognin

1 Introduction

1.1 Settings

All the following experiments and development (e.g., training, testing) were conducted on the same hardware, which includes an Nvidia RTX 3070 GPU, an Intel i9-10900 CPU (2.9 GHz), and 32 GB of RAM. The primary Python libraries utilized are Torch and Scikit-learn. The virtual environment was managed using uv.

2 Use of LLMs

Within the scope of this laboratory, LLMs were used as an aid for translating texts into English and for looking up specific functions/attributes in the libraries employed in the project.

2.1 Exercise 1

In the preliminary data analysis, the dataset structure was verified by consulting its official page on **HuggingFace** (`cornell-movie-review-data/rotten_tomatoes`), followed by a detailed exploration of its distribution properties. The label distribution is perfectly balanced, exhibiting an equal number of samples for each class across all dataset splits. Conversely, the text length demonstrates high variability, ranging from a maximum sequence length of **59 characters** to a minimum of **1 character**.

By analyzing the most frequent words within each label group, specific linguistic patterns predictive of class membership can be identified a priori. After filtering out standard stop words, the remaining top frequencies were examined. For the positive review class, the most frequent terms include `'most', 'will', 'comedy', 'good', 'funny', 'best', 'time', 'up', 'much', 'love'`, whereas the negative reviews are characterized by prominent keywords such as `'bad', 'little', 'characters', '?', 'most', 'when', 'never', 'would'`.

2.2 Exercise 1.3

In this exercise, using a pipeline dedicated to feature extraction by the `distilbert/distilbert-base-uncased` model, we extract the features of the CLS token associated with each element of the dataset passed through the model and feed them to the SVC classifier. After fitting the classifier on the training dataset, it is validated on the validation split to identify the best model parameters.

Best Model Parameters:

- `C`: 10
- `kernel`: `rbf`

Table 1: Classification Report (Validation)

	Precision	**Recall**	**F1-score**	**Support**
0	0.80	0.85	0.83	533
1	0.84	0.79	0.81	533
Accuracy			0.82	1066
Macro Avg	0.82	0.82	0.82	1066
Weighted Avg	0.82	0.82	0.82	1066

Table 2: Classification Report (Test Set)

	Precision	**Recall**	**F1-score**	**Support**
0	0.79	0.83	0.81	533
1	0.82	0.78	0.80	533
Accuracy			0.80	1066
Macro Avg	0.80	0.80	0.80	1066
Weighted Avg	0.80	0.80	0.80	1066

The test results indicate a good classifier capable of identifying the correct classes, even though the backbone used was not fine-tuned for the dataset in question. Furthermore, the parameters selected for the model do not lead to overfitting, as the testing results do not deviate significantly from those obtained during the validation phase.

3 Exercise 2

After tokenizing and adapting the textual data from the training, evaluation, and testing datasets, we proceed to define the `TrainingArguments` and the

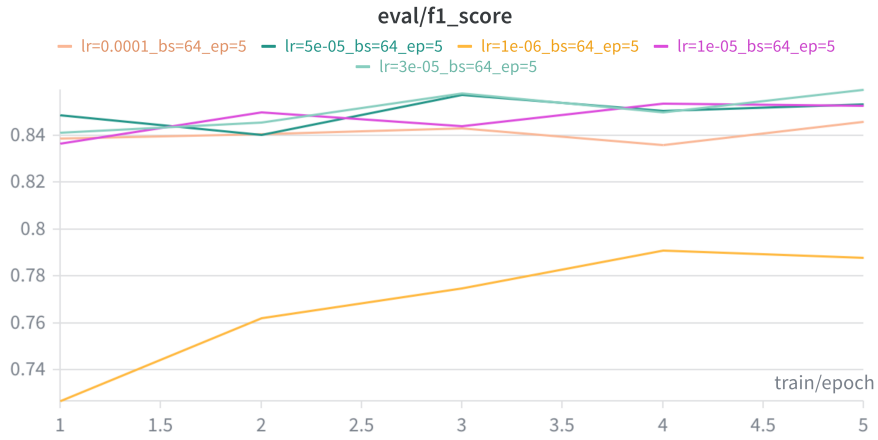

Figure 1: F1-score across different learning rates

trainer. At this stage, `AutoModelForSequenceClassification` was used to instantiate the appropriate model for the task to be performed on the dataset. Numerous possible model configurations were then compared to select the most suitable one based on the final results of the validation metrics.

The chosen evaluation metric took into account precision, recall, F1-score, loss, and accuracy. By observing the metric plots on Weights & Biases, it is evident that the best-performing model is the one using a learning rate $\eta = 1 \times 10^{-5}$ and a `batch_size` of 32 during training, with model checkpoints saved based on the best F1-score value, which was then evaluated on the test set.

The choice of the reference metric is dictated by the classification task and the fact that the dataset is balanced. However, it remains clear that for most configurations, the model quickly overfits within a few epochs, showing a very pronounced increase in validation loss at each epoch, except in the case of $\eta = 1 \times 10^{-5}$. Other smaller learning rates maintain a similar convergence behavior, yet the overall model performance remains analogous.

4 Exercise 3.3

4.1 Implementation details

To implement the image retrieval application, the `openai/clip-vit-base-patch32` model was used. Due to its multimodal nature and its ability to project both image and text features into the same embedding space, the CLIP model is excellent for performing image retrieval tasks. The reference model can be modified via the configuration file. The `jxie/flickr8k` dataset was selected to provide the images upon which the retrieval process is executed.

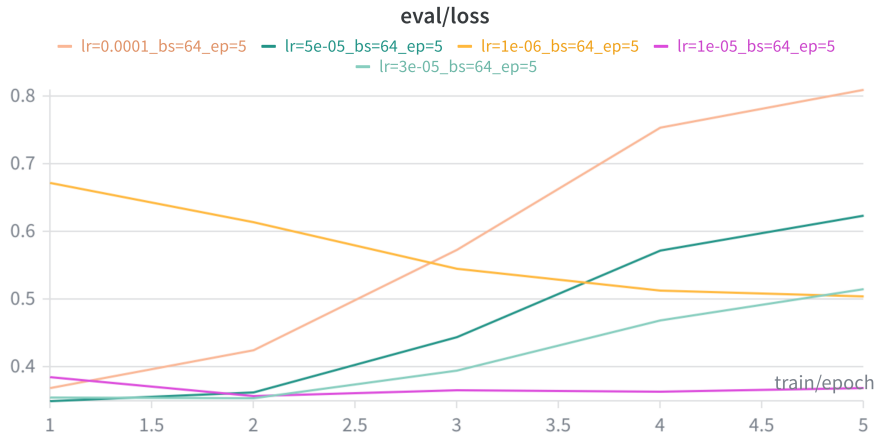

Figure 2: Loss across different learning rates

4.2 Text-to-Image retrieval

During the first run of the application, the image features from the dataset are computed by the model and saved so they can be loaded for future requests, thereby reducing computational load and execution time. The similarity between the image features and the textual features is computed using cosine similarity. Based on this similarity score, the system outputs the top 10 images with the highest cosine similarity relative to the input prompt.

4.3 Interface

The user interface was implemented using `gradio`. The application consists of a text box where the input prompt can be entered, a section dedicated to displaying the output images generated by the model, and a button to initiate the retrieval process.